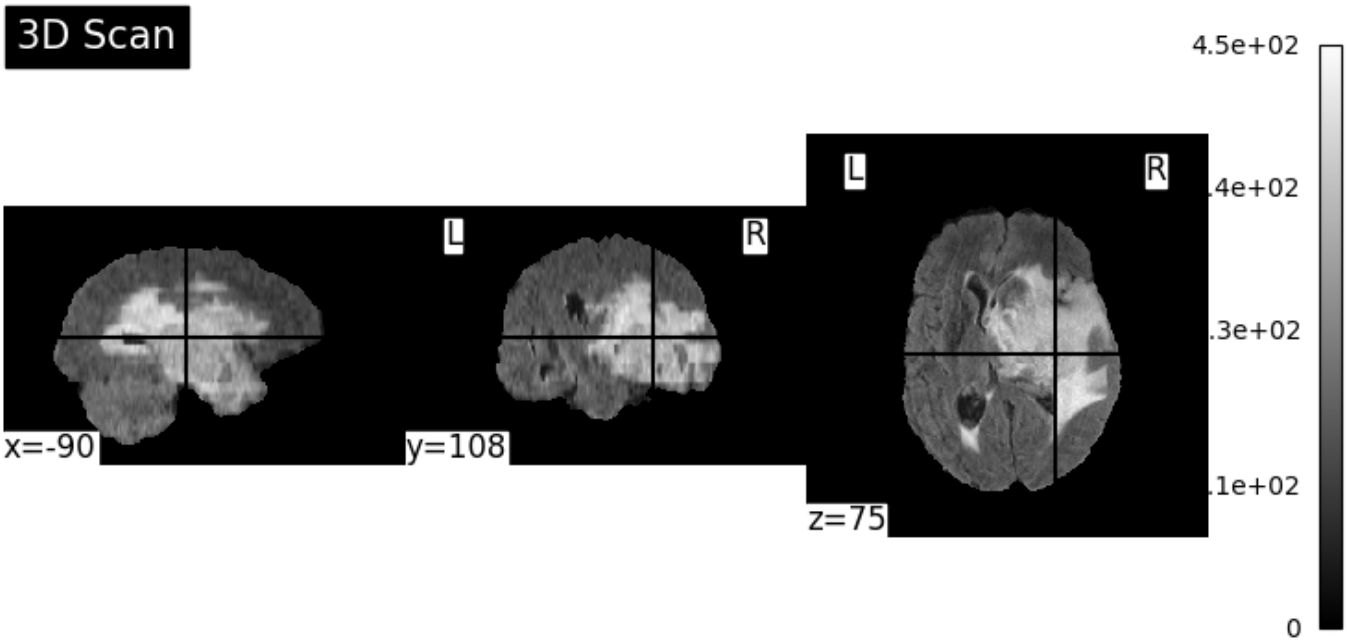
In [1]:

```
from preprocess.conversion import visualise_datapoint, visualise_scan_data, convert_all_
```

To visualise the samples, I have created several helper functions in preprocess/conversion.py. We can visualise a single datapoint by calling visualise_scan_data(). We can also visualise all four, by calling visualise_datapoint().

In [2]:

```
datapoints = []

datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_flair.nii")
datapoints.append("\BraTS20_Training_001\BraTS20_Training_001_seg.nii")

for datapoint in datapoints:
    visualise_scan_data(datapoint, is_train=True)
```

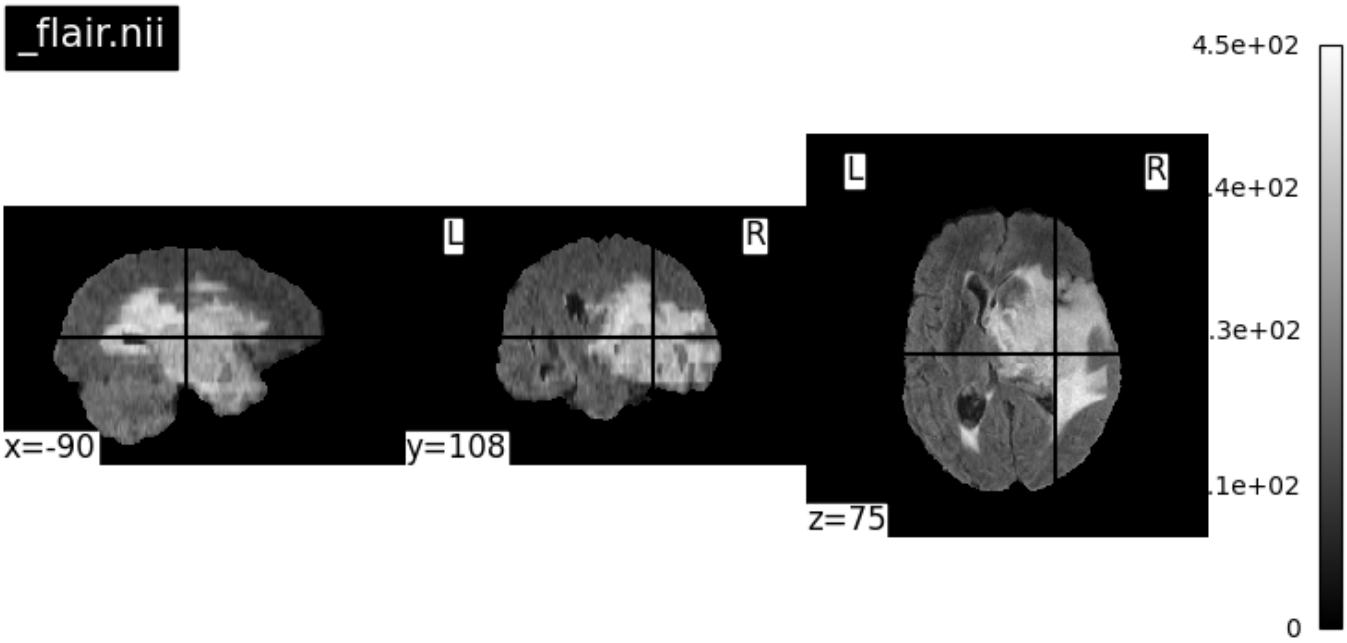


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

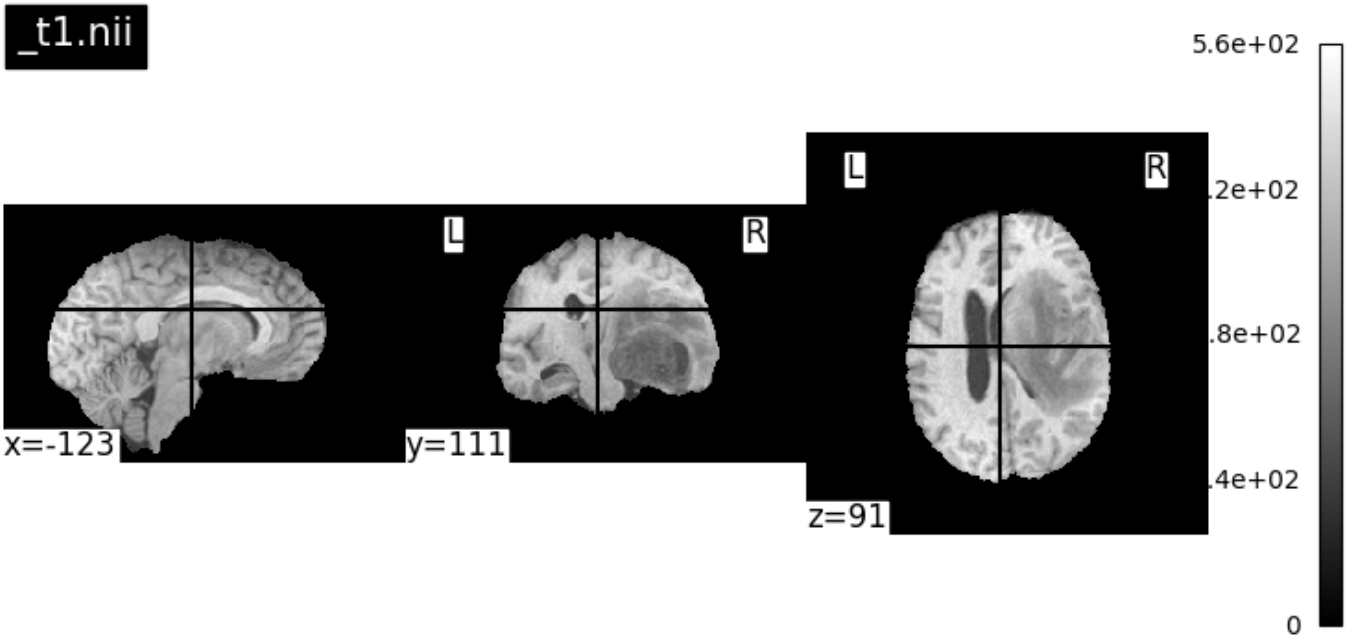
To visualise all four parts, we can use:

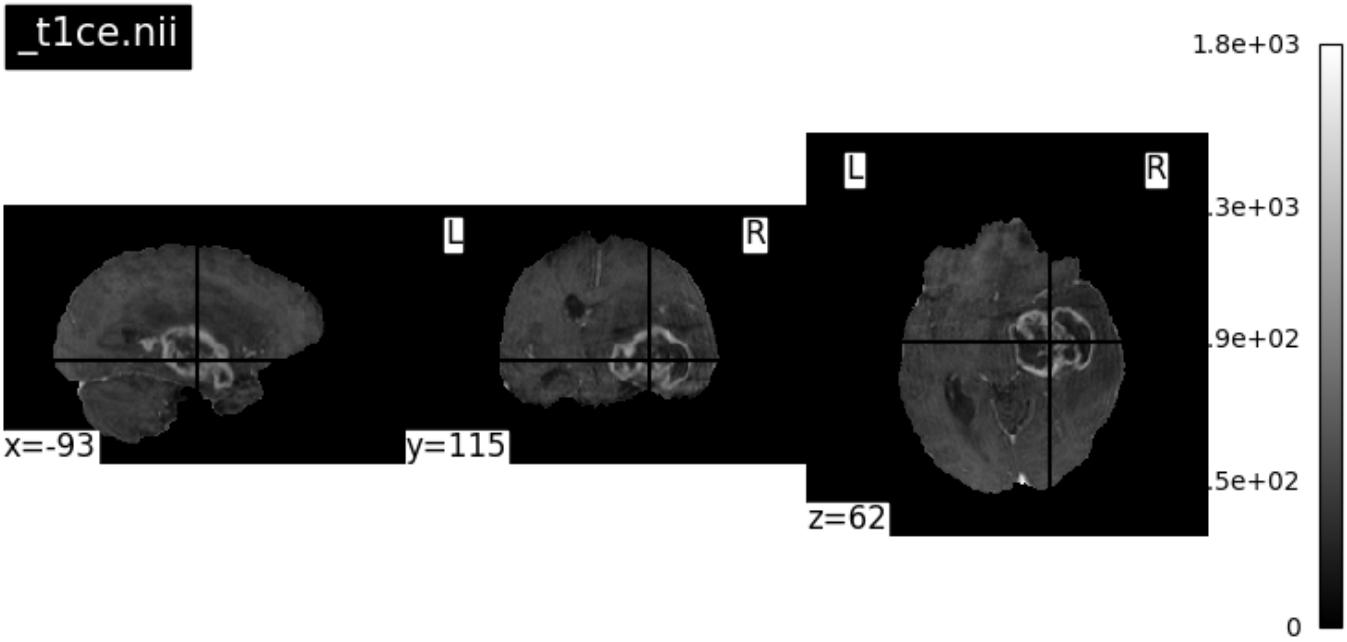
In [3]:

```
path = "\BraTS20_Training_001"
visualise_datapoint(path, is_train=True)
```

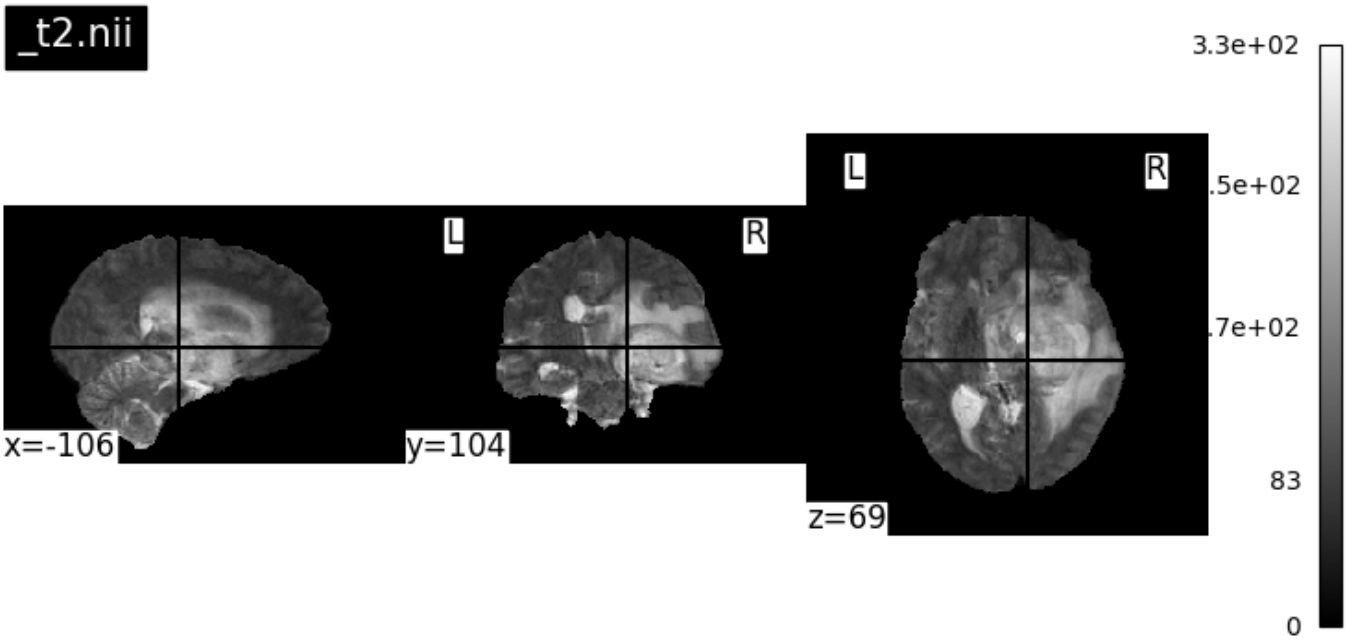


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Before training the model, all .nii.gz files should be converted into NumPy arrays and saved locally to significantly reduce I/O overhead during training, since loading compressed medical imaging files repeatedly is slow. As part of this conversion process, each scan must be preprocessed to ensure consistency across the dataset: first, remove empty background regions by cropping to the smallest bounding box containing meaningful information, which reduces memory usage and computational cost. then, we apply scaling using an appropriate scaler, in this case we used z-score normalisation to normalize intensities and stabilize model training.

In [2]:

```
convert_all_npy()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[2], line 1
----> 1 convert_all_npy()

NameError: name 'convert_all_npy' is not defined
```

Now that we have completed the converting all the files into numpy format, we have to split the dataset into a training set and a testing set. If we look above, we can see that the t1 image does not look like it provides much useful information. Therefore, it will be skipped and not added into our training pipeline. Now we need to load our data into main memory.

In [1]:

```
from preprocess.splitting import train_test_split, combine_from_npy

features, labels = combine_from_npy()
```

In [2]:

```
features.shape
```

Out[2]:

```
(220, 3, 128, 128, 128)
```

In [3]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
labels.shape
```

Out[3]:
```
(220, 4, 128, 128, 128)
```

In [4]:
```
import torch
if torch.cuda.is_available():
    device = torch.device('cuda')
    print("Using cuda")
else:
    device = torch.device('cpu')
    print("Using cpu")
```

```
Using cuda
```

In [5]:
```
train_dataloader, test_dataloader = train_test_split(features, labels, device)
del features
del labels
```

```
Training batches: 88
Testing batches: 22
```

In [6]:
```
from models.UNet.unet import UNet
from models.UNet.unet_attn import AttentionUNet
from models.UNet.params import UNetParams

params = UNetParams()
#model = UNet(params=params).to(device=device)
model = AttentionUNet(params=params).to(device=device)
```

In [7]:
```
from utility.utils import init_weights

model.apply(init_weights)
```

Out[7]:
```
AttentionUNet(
  (down_conv1): ConvReLUBlock(
    (conv3d_1): Conv3d(3, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
  )
  (down_conv2): ConvReLUBlock(
    (conv3d_1): Conv3d(16, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
  )
  (down_conv3): ConvReLUBlock(
    (conv3d_1): Conv3d(32, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
1))
    (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
  )
  (down_conv4): ConvReLUBlock(
    (conv3d_1): Conv3d(64, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
  )
  (bottlenck): ConvReLUBlock(
    (conv3d_1): Conv3d(128, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (conv3d_2): Conv3d(256, 256, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
    (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
    (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
    (grp_norm): GroupNorm(8, 256, eps=1e-05, affine=True)
  )
  (maxpool): MaxPool3d(kernel_size=(2, 2, 2), stride=(2, 2, 2), padding=0, dilation=1, c
eil_mode=False)
  (up_conv1_attn): ConvTransposeAttn(
    (conv3d_transposed): ConvTranspose3d(256, 128, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(256, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (conv3d_2): Conv3d(128, 128, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 128, eps=1e-05, affine=True)
    )
    (attn_applied): AttnModule(
      (guidance_signal): Sequential(
        (0): Conv3d(128, 128, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 128, eps=1e-05, affine=True)
      )
      (skip_signal): Sequential(
        (0): Conv3d(128, 128, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 128, eps=1e-05, affine=False)
      )
      (attn_map): Sequential(
        (0): Conv3d(128, 128, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): Sigmoid()
      )
      (relu): ReLU(inplace=True)
    )
    (groupnorm): GroupNorm(8, 256, eps=1e-05, affine=False)
  )
  (up_conv2_attn): ConvTransposeAttn(
    (conv3d_transposed): ConvTranspose3d(128, 64, kernel_size=(2, 2, 2), stride=(2, 2,
2))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(128, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1,
1, 1))
      (conv3d_2): Conv3d(64, 64, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 64, eps=1e-05, affine=True)
    )
    (attn_applied): AttnModule(
      (guidance_signal): Sequential(
        (0): Conv3d(64, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 64, eps=1e-05, affine=True)
      )
      (skip_signal): Sequential(
        (0): Conv3d(64, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 64, eps=1e-05, affine=False)
      )
      (attn_map): Sequential(
        (0): Conv3d(64, 64, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): Sigmoid()
      )
      (relu): ReLU(inplace=True)
    )
    (groupnorm): GroupNorm(8, 128, eps=1e-05, affine=False)
  )
  (up_conv3_attn): ConvTransposeAttn(
    (conv3d_transposed): ConvTranspose3d(64, 32, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
      (conv3d_1): Conv3d(64, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (conv3d_2): Conv3d(32, 32, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 32, eps=1e-05, affine=True)
    )
    (attn_applied): AttnModule(
      (guidance_signal): Sequential(
        (0): Conv3d(32, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 32, eps=1e-05, affine=True)
      )
      (skip_signal): Sequential(
        (0): Conv3d(32, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 32, eps=1e-05, affine=False)
      )
      (attn_map): Sequential(
        (0): Conv3d(32, 32, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): Sigmoid()
      )
      (relu): ReLU(inplace=True)
    )
    (groupnorm): GroupNorm(8, 64, eps=1e-05, affine=False)
  )
  (up_conv4_attn): ConvTransposeAttn(
    (conv3d_transposed): ConvTranspose3d(32, 16, kernel_size=(2, 2, 2), stride=(2, 2,
2))
    (conv3d_block): ConvReLUBlock(
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
      (conv3d_1): Conv3d(32, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (conv3d_2): Conv3d(16, 16, kernel_size=(3, 3, 3), stride=(1, 1, 1), padding=(1, 1,
1))
      (relu1): LeakyReLU(negative_slope=0.01, inplace=True)
      (relu2): LeakyReLU(negative_slope=0.01, inplace=True)
      (grp_norm): GroupNorm(8, 16, eps=1e-05, affine=True)
    )
    (attn_applied): AttnModule(
      (guidance_signal): Sequential(
        (0): Conv3d(16, 16, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 16, eps=1e-05, affine=True)
      )
      (skip_signal): Sequential(
        (0): Conv3d(16, 16, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): GroupNorm(8, 16, eps=1e-05, affine=False)
      )
      (attn_map): Sequential(
        (0): Conv3d(16, 16, kernel_size=(1, 1, 1), stride=(1, 1, 1))
        (1): Sigmoid()
      )
      (relu): ReLU(inplace=True)
    )
    (groupnorm): GroupNorm(8, 32, eps=1e-05, affine=False)
  )
  (final_conv): Conv3d(16, 4, kernel_size=(1, 1, 1), stride=(1, 1, 1))
)
```

In [8]:

```
import torch._dynamo
print(torch._dynamo.list_backends())
```

```
['cudagraphs', 'inductor', 'openxla', 'tvm']
```

In [9]:

```
model = torch.compile(model=model, backend="inductor")
```

In [10]:

```
from tqdm import tqdm

from utility.utils import train_loop, inference_loop
from utility.metrics import DiceCELoss
epochs = 70

torch.backends.cudnn.benchmark = True
torch.set_float32_matmul_precision("high")

train_losses = []
test_losses = []
train_dice_scores = []
test_dice_scores = []

optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode='min',
    factor=0.5,
    patience=10,
    min_lr=1e-6
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
)
loss = DiceCELoss().to(device=device)

for epoch in tqdm(range(1, epochs+1)):

    train_epoch_loss, train_dice_score = train_loop(
        model=model,
        train_dataloader=train_dataloader,
        loss_fn=loss,
        optimizer=optimizer,
        device=device
    )
    test_epoch_loss, test_dice_score = inference_loop(
        model=model,
        test_dataloader=test_dataloader,
        loss_fn=loss,
        device=device
    )

    print(f"training loss : {train_epoch_loss}")
    print(f"testing loss : {test_epoch_loss}")

    train_losses.append(train_epoch_loss)
    test_losses.append(test_epoch_loss)
    train_dice_scores.append(train_dice_score)
    test_dice_scores.append(test_dice_score)
```

```
  0%|          | 0/70 [00:00<?, ?it/s]
W0313 07:37:52.306000 4888 site-packages\torch\_inductor\utils.py:1613] [0/0] Not enough
SMs to use max_autotune_gemm mode
Training Loop: 88it [07:46,  5.30s/it]
Evaluation Loop: 22it [00:48,  2.20s/it]
  1%|          | 1/70 [08:37<9:54:37, 517.06s/it]
training loss : 2.6232054937969553
testing loss : 1.868947608904405
Training Loop: 88it [06:46,  4.62s/it]
Evaluation Loop: 22it [00:25,  1.17s/it]
  3%|          | 2/70 [15:49<8:49:26, 467.15s/it]
training loss : 1.8421502072702756
testing loss : 1.7335312962532043
Training Loop: 88it [06:42,  4.58s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
  4%|          | 3/70 [22:56<8:21:12, 448.84s/it]
training loss : 1.7120879780162463
testing loss : 1.6417569626461377
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
  6%|          | 4/70 [29:31<7:50:18, 427.56s/it]
training loss : 1.6117789515040137
testing loss : 1.5451095700263977
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
  7%|          | 5/70 [36:06<7:30:25, 415.77s/it]
training loss : 1.4909625757824292
testing loss : 1.4455169385129756
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
  9%|          | 6/70 [42:41<7:15:54, 408.67s/it]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
training loss : 1.404609588059512
testing loss : 1.4095735604112798
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 10%|▉         | 7/70 [49:15<7:04:14, 404.04s/it]
training loss : 1.321141543713483
testing loss : 1.2923749956217678
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 11%|▉         | 8/70 [55:49<6:54:12, 400.84s/it]
training loss : 1.245679333806038
testing loss : 1.2538798397237605
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 13%|▉         | 9/70 [1:02:24<6:45:48, 399.15s/it]
training loss : 1.202730350873687
testing loss : 1.2012136632745916
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 14%|▉         | 10/70 [1:08:59<6:37:45, 397.76s/it]
training loss : 1.1724188950928776
testing loss : 1.2121122723275966
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 16%|▉         | 11/70 [1:15:34<6:30:22, 396.99s/it]
training loss : 1.161165328188376
testing loss : 1.2107809782028198
Training Loop: 88it [06:10,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 17%|▉         | 12/70 [1:22:10<6:23:14, 396.46s/it]
training loss : 1.1365123682401397
testing loss : 1.1473384770480068
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 19%|▉         | 13/70 [1:28:45<6:16:17, 396.10s/it]
training loss : 1.1027663011442532
testing loss : 1.1291294287551532
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 20%|▉         | 14/70 [1:35:20<6:09:24, 395.80s/it]
training loss : 1.065900138833306
testing loss : 1.1505275802178816
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 21%|▉         | 15/70 [1:41:55<6:02:44, 395.71s/it]
training loss : 1.0821280255913734
testing loss : 1.0729720186103473
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 23%|▉         | 16/70 [1:48:30<5:55:47, 395.32s/it]
training loss : 1.094066790559075
testing loss : 1.2101782668720593
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 24%|▉         | 17/70 [1:55:05<5:49:05, 395.20s/it]
training loss : 1.0777289413593032
testing loss : 1.0848146677017212
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
 26%|██        | 18/70 [2:01:40<5:42:28, 395.17s/it]
training loss : 1.008954400366003
testing loss : 1.0497510406104
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 27%|██        | 19/70 [2:08:15<5:35:55, 395.20s/it]
training loss : 1.003456142138351
testing loss : 1.1685758341442456
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 29%|██        | 20/70 [2:14:51<5:29:24, 395.28s/it]
training loss : 1.0548358674753795
testing loss : 1.1348935067653656
Training Loop: 88it [06:10,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 30%|██        | 21/70 [2:21:26<5:22:47, 395.25s/it]
training loss : 1.0426350506869229
testing loss : 1.110700230706822
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 31%|██        | 22/70 [2:28:02<5:16:22, 395.46s/it]
training loss : 1.0211816775527867
testing loss : 1.0011465441096912
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 33%|██        | 23/70 [2:34:37<5:09:43, 395.40s/it]
training loss : 0.9416514458981428
testing loss : 1.02754744887352
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 34%|██        | 24/70 [2:41:13<5:03:14, 395.53s/it]
training loss : 0.931437687440352
testing loss : 0.9538345336914062
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 36%|██        | 25/70 [2:47:48<4:56:28, 395.31s/it]
training loss : 0.8921876346523111
testing loss : 0.9447629235007546
Training Loop: 88it [06:14,  4.26s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 37%|██        | 26/70 [2:54:27<4:50:43, 396.44s/it]
training loss : 0.873254498297518
testing loss : 0.9144694263284857
Training Loop: 88it [06:12,  4.23s/it]
Evaluation Loop: 22it [00:23,  1.07s/it]
 39%|██        | 27/70 [3:01:03<4:44:03, 396.37s/it]
training loss : 0.8465407924218611
testing loss : 0.940801354971799
Training Loop: 88it [06:09,  4.20s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 40%|██        | 28/70 [3:07:37<4:37:00, 395.72s/it]
training loss : 0.8668768344955011
testing loss : 0.9260168644514951
Training Loop: 88it [06:14,  4.25s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 41%|██        | 29/70 [3:14:16<4:30:59, 396.57s/it]
training loss : 0.8206634216687896
testing loss : 0.928489855744622
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Training Loop: 88it [06:09,  4.20s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 43%|▌▌▌▌      | 30/70 [3:20:50<4:23:53, 395.83s/it]
training loss : 0.8254260288720782
testing loss : 0.8781607882543043
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 44%|▌▌▌▌      | 31/70 [3:27:25<4:17:06, 395.56s/it]
training loss : 0.8115301843393933
testing loss : 0.8739054338498549
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:23,  1.06s/it]
 46%|▌▌▌▌      | 32/70 [3:34:00<4:10:27, 395.46s/it]
training loss : 0.7981078753417189
testing loss : 0.8967173533006148
Training Loop: 88it [06:17,  4.29s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 47%|▌▌▌▌      | 33/70 [3:40:42<4:05:04, 397.42s/it]
training loss : 0.82005979323929
testing loss : 0.9391471987420862
Training Loop: 88it [06:12,  4.24s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 49%|▌▌▌▌      | 34/70 [3:47:19<3:58:23, 397.31s/it]
training loss : 0.793846724385565
testing loss : 0.8252676562829451
Training Loop: 88it [06:09,  4.20s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 50%|▌▌▌▌      | 35/70 [3:53:52<3:51:05, 396.14s/it]
training loss : 0.7737593705003912
testing loss : 0.8610601912845265
Training Loop: 88it [06:12,  4.23s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 51%|▌▌▌▌      | 36/70 [4:00:28<3:44:26, 396.07s/it]
training loss : 0.7752717889168046
testing loss : 0.8952903151512146
Training Loop: 88it [06:09,  4.20s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 53%|▌▌▌▌      | 37/70 [4:07:01<3:37:21, 395.20s/it]
training loss : 0.8438061733137477
testing loss : 0.8891914866187356
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 54%|▌▌▌▌      | 38/70 [4:13:37<3:30:44, 395.15s/it]
training loss : 0.8120071058246222
testing loss : 0.9723088212988593
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 56%|▌▌▌▌      | 39/70 [4:20:11<3:24:04, 394.99s/it]
training loss : 0.78809600120241
testing loss : 0.9151420376517556
Training Loop: 88it [06:17,  4.29s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 57%|▌▌▌▌      | 40/70 [4:26:53<3:18:31, 397.06s/it]
training loss : 0.774491632526571
testing loss : 0.8596195158633319
Training Loop: 88it [06:09,  4.20s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 59%|▌▌▌▌      | 41/70 [4:33:27<3:11:28, 396.16s/it]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
training loss : 0.7537235465239395
testing loss : 0.8586439029736952
Training Loop: 88it [06:14,  4.25s/it]
Evaluation Loop: 22it [00:24,  1.11s/it]
 60%|▉▉▉
```

```
 76%|▉▉▉▉▉▉▉▉   | 53/70 [5:52:35<1:51:57, 395.15s/it]
training loss : 0.6128841509873216
testing loss : 0.7450342638926073
Training Loop: 88it [06:13,  4.25s/it]
Evaluation Loop: 22it [00:23,  1.07s/it]
 77%|▉▉▉▉▉▉▉▉   | 54/70 [5:59:13<1:45:33, 395.86s/it]
training loss : 0.5992993092672392
testing loss : 0.7705097266218879
Training Loop: 88it [06:12,  4.23s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 79%|▉▉▉▉▉▉▉▉   | 55/70 [6:05:49<1:38:59, 396.00s/it]
training loss : 0.601825434375893
testing loss : 0.7815711295062845
Training Loop: 88it [06:16,  4.28s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 80%|▉▉▉▉▉▉▉▉   | 56/70 [6:12:30<1:32:43, 397.37s/it]
training loss : 0.6219345564869317
testing loss : 0.7475936805660074
Training Loop: 88it [06:08,  4.19s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 81%|▉▉▉▉▉▉▉▉   | 57/70 [6:19:02<1:25:48, 396.01s/it]
training loss : 0.613717586140741
testing loss : 0.7734555507248099
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 83%|▉▉▉▉▉▉▉▉   | 58/70 [6:25:38<1:19:10, 395.83s/it]
training loss : 0.6292691278186712
testing loss : 0.7095839327031915
Training Loop: 88it [06:14,  4.26s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 84%|▉▉▉▉▉▉▉▉   | 59/70 [6:32:17<1:12:44, 396.73s/it]
training loss : 0.6082063696601174
testing loss : 0.7370686151764609
Training Loop: 88it [06:12,  4.23s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 86%|▉▉▉▉▉▉▉▉   | 60/70 [6:38:53<1:06:06, 396.67s/it]
training loss : 0.6005919579077851
testing loss : 0.7406735135750337
Training Loop: 88it [06:08,  4.19s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 87%|▉▉▉▉▉▉▉▉   | 61/70 [6:45:26<59:20, 395.59s/it]
training loss : 0.5847219323570078
testing loss : 0.6954721104015004
Training Loop: 88it [06:16,  4.28s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 89%|▉▉▉▉▉▉▉▉   | 62/70 [6:52:07<52:57, 397.23s/it]
training loss : 0.5607877803797071
testing loss : 0.7305029156533155
Training Loop: 88it [06:08,  4.19s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 90%|▉▉▉▉▉▉▉▉   | 63/70 [6:58:40<46:11, 395.94s/it]
training loss : 0.5669806220314719
testing loss : 0.7873208604075692
Training Loop: 88it [06:10,  4.21s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 91%|▉▉▉▉▉▉▉▉   | 64/70 [7:05:15<39:33, 395.56s/it]
training loss : 0.6482563127170909
testing loss : 0.7067183147777211
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
Training Loop: 88it [06:15,  4.27s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
 93%|▉▉▉▉▉▉▉▉▉▉| 65/70 [7:11:55<33:04, 396.81s/it]
training loss : 0.6053933125327934
testing loss : 0.6923512410033833
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 94%|▉▉▉▉▉▉▉▉▉▉| 66/70 [7:18:30<26:25, 396.48s/it]
training loss : 0.6041248298504136
testing loss : 0.7203857668421485
Training Loop: 88it [06:08,  4.19s/it]
Evaluation Loop: 22it [00:24,  1.10s/it]
 96%|▉▉▉▉▉▉▉▉▉▉| 67/70 [7:25:03<19:46, 395.39s/it]
training loss : 0.6179805885661732
testing loss : 0.751949658448046
Training Loop: 88it [06:11,  4.22s/it]
Evaluation Loop: 22it [00:23,  1.06s/it]
 97%|▉▉▉▉▉▉▉▉▉▉| 68/70 [7:31:38<13:10, 395.30s/it]
training loss : 0.6056407764554024
testing loss : 0.714634048667821
Training Loop: 88it [06:10,  4.22s/it]
Evaluation Loop: 22it [00:24,  1.09s/it]
 99%|▉▉▉▉▉▉▉▉▉▉| 69/70 [7:38:13<06:35, 395.21s/it]
training loss : 0.6006128276613626
testing loss : 0.773867894302715
Training Loop: 88it [06:20,  4.33s/it]
Evaluation Loop: 22it [00:23,  1.08s/it]
100%|▉▉▉▉▉▉▉▉▉▉| 70/70 [7:44:58<00:00, 398.55s/it]
training loss : 0.6476753652095795
testing loss : 0.8482120443474163
```

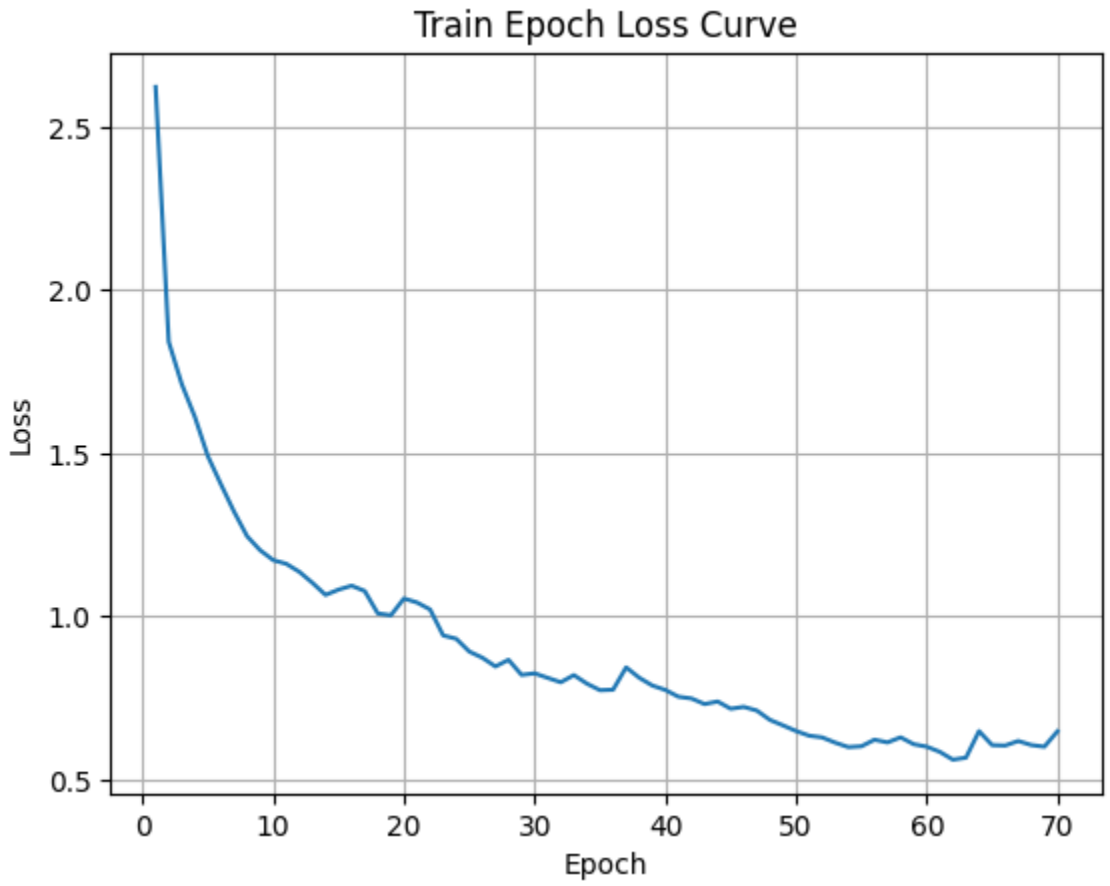
In [11]:

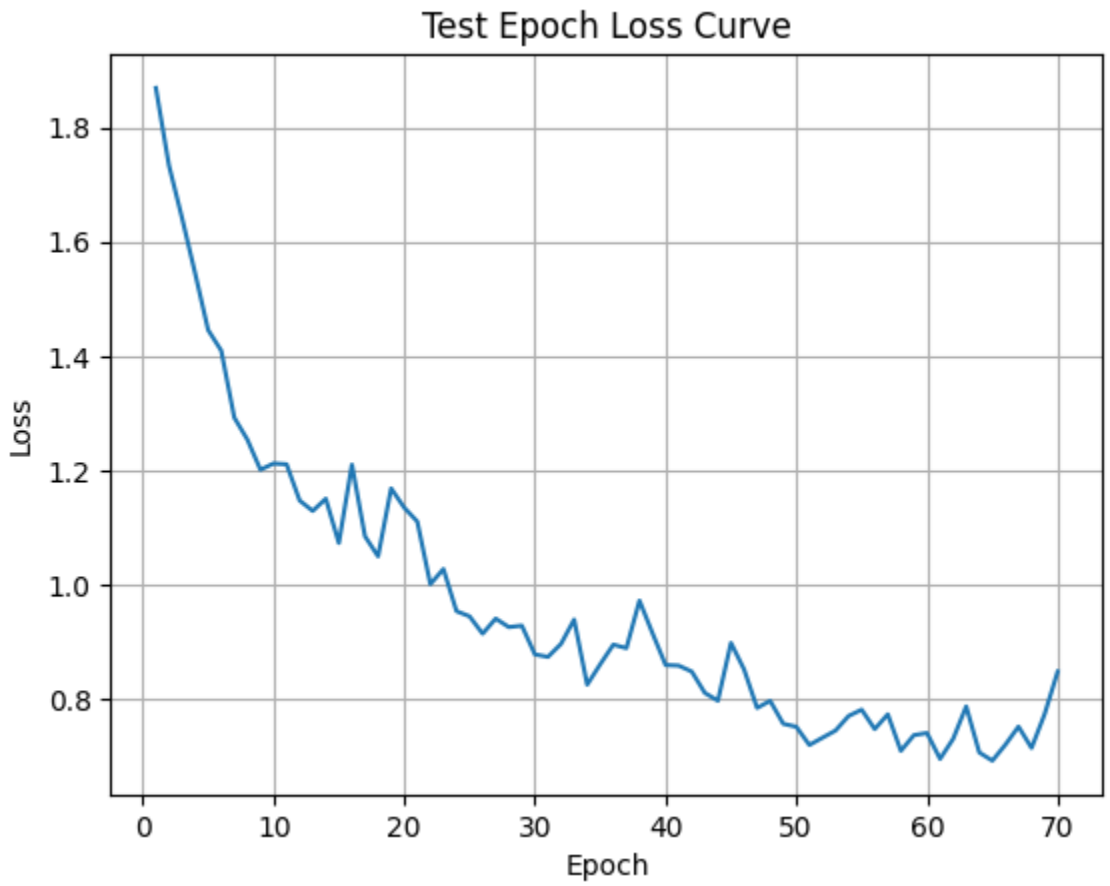
```
torch.save(model.state_dict(), "unet3d_4.1.pth")
del model
del train_dataloader
del test_dataloader
```

In [12]:

```
from utility.plotting import plot_graph
plot_graph(train_losses, "Train Epoch Loss Curve", "Epoch", "Loss")
plot_graph(test_losses, "Test Epoch Loss Curve", "Epoch", "Loss")
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



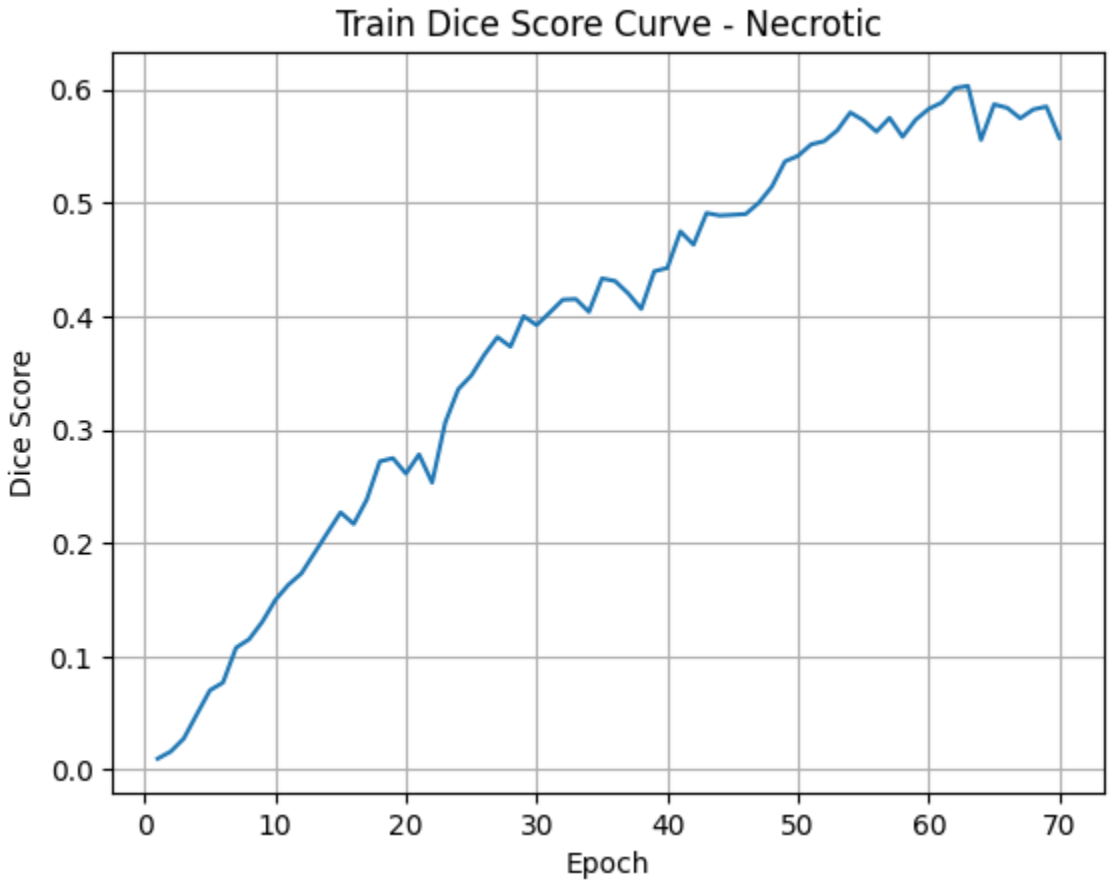


In [13]:

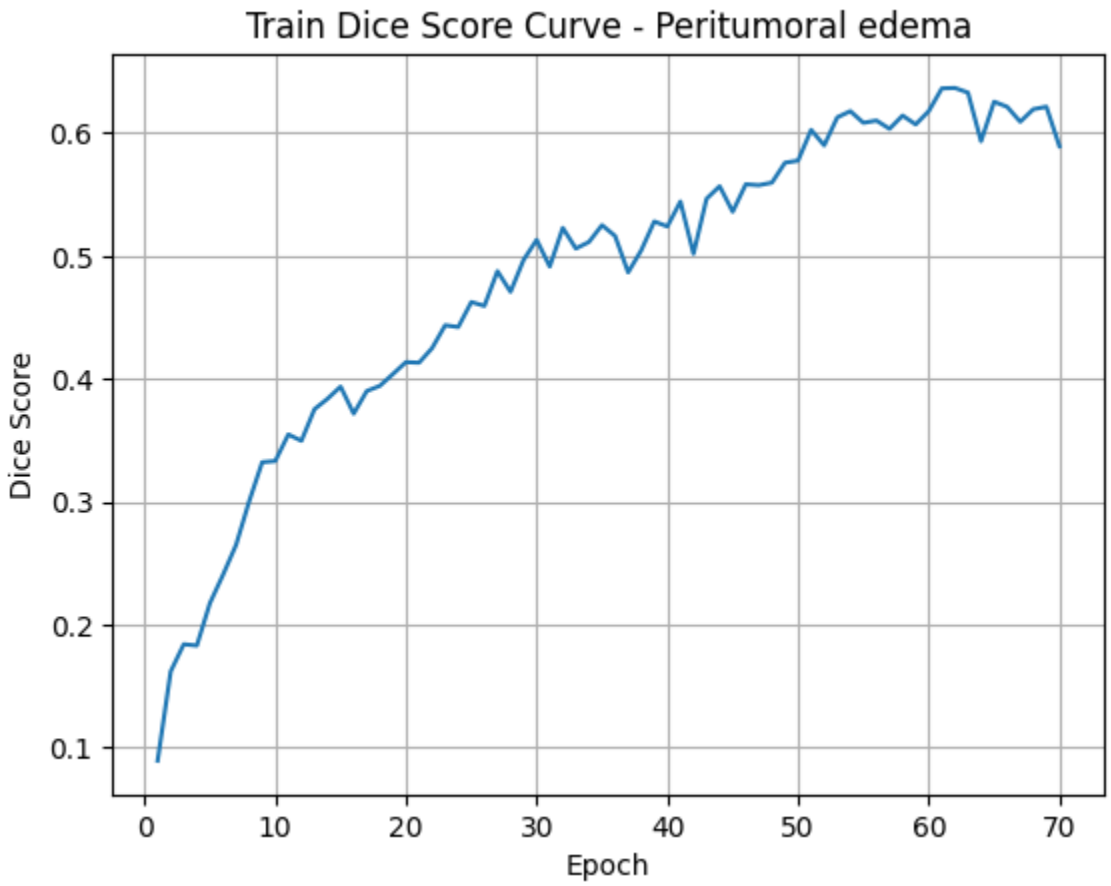
```
plot_graph([t.detach().cpu()[1] for t in train_dice_scores], "Train Dice Score Curve - N
plot_graph([t.detach().cpu()[1] for t in test_dice_scores], "Test Dice Score Curve - Nec
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

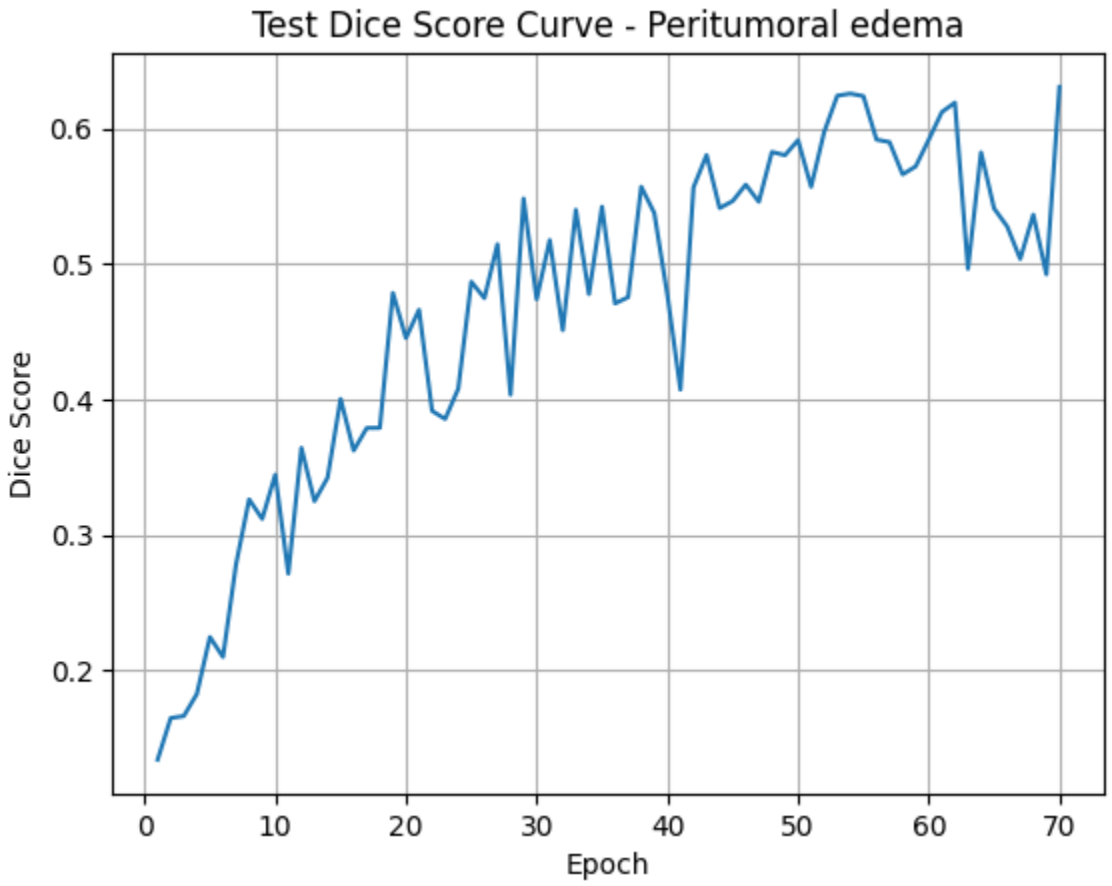
```
plot_graph([t.detach().cpu()[2] for t in train_dice_scores], "Train Dice Score Curve - P
plot_graph([t.detach().cpu()[2] for t in test_dice_scores], "Test Dice Score Curve - Per

plot_graph([t.detach().cpu()[3] for t in train_dice_scores], "Train Dice Score Curve - G
plot_graph([t.detach().cpu()[3] for t in test_dice_scores], "Test Dice Score Curve - GD-
```

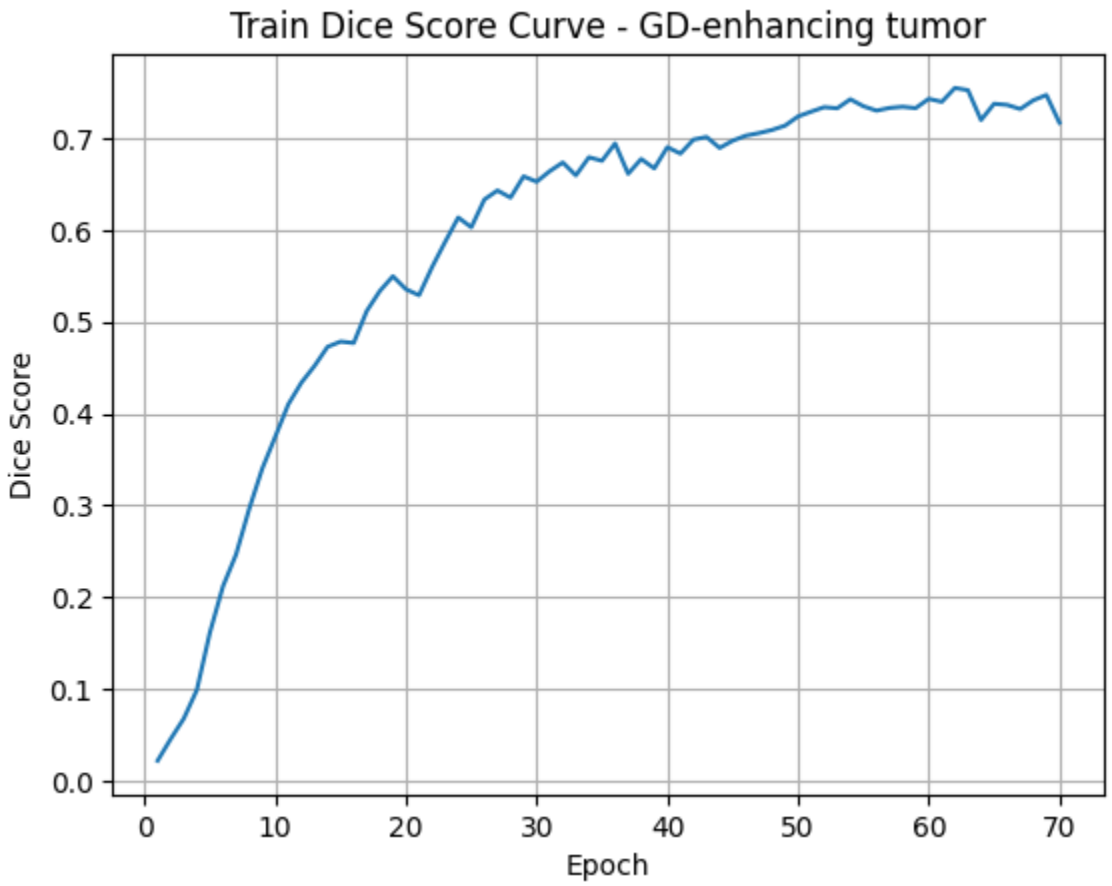


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

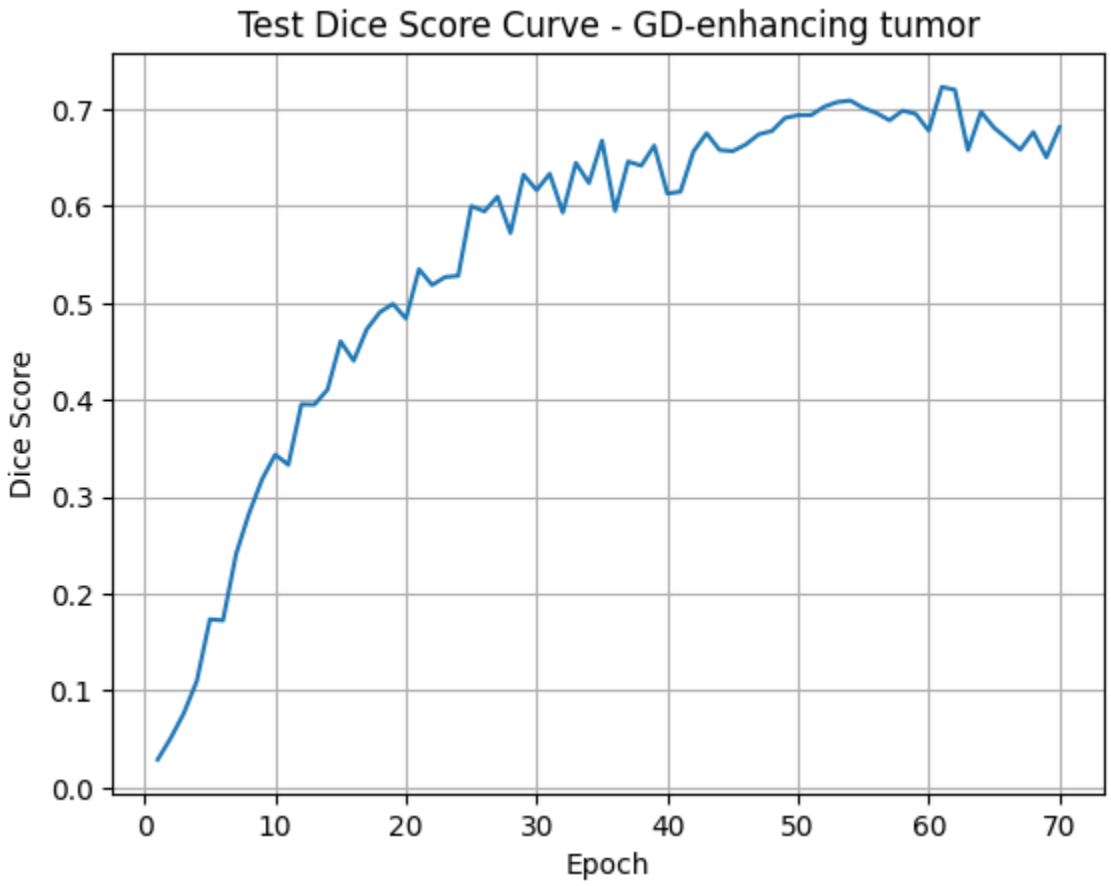


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF





Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF